議論メモ：虚音源（鏡像）の式の `abs()` について

作成: 2026-08-14（HBN240018）／最終確認: 2026-08-23

要判断のまま（TODO E-12）。暫定で符号付きを採用して計算している。 この件は 2026-08-14 以降変わっていないので、本文もそのままでよい。

対応コード: `geosim/loop_noredundancy.py` の `image_sources()` 元コード: `backtrace.f90` 907〜917 行

---

0. 結論だけ先に

元コードは鏡像を作るときに**距離の絶対値**を使っています。

```
temp = abs(n・p + d) / |n|              ! ← abs がついている
isrc(k,l) = isrc(k-1,l) - 2.0 * temp * n(l)
```

これは「**虚音源が面の表側にあるとき**」しか正しくありません。 裏側にあると、面を跨いで反対側に移すはずが、**面から遠ざかる方向へ動いてしまいます。**

Python 版は `abs()` を外した符号付きの式にしてあります。

```
signed_distance = np.dot(normal, images[k]) + d      # 絶対値を取らない
images[k + 1] = images[k] - 2.0 * signed_distance * normal
```

凸な部屋（直方体など）では両者は完全に一致します。 差が出るのは凹んだ形状・ 宙に浮いた物体・法線を反転させたモデルなどです。

---

1. そもそも鏡像とは何をしているか

1.1 平面の方程式と「符号付き距離」

面の単位法線を **n**、面上の 1 点を $\mathbf{x}_1$ とすると、その面を含む平面は

$$\mathbf{n}\cdot\mathbf{x} + d = 0, \qquad d = -\,\mathbf{n}\cdot\mathbf{x}_1$$

と書けます。ここで任意の点 **p** について

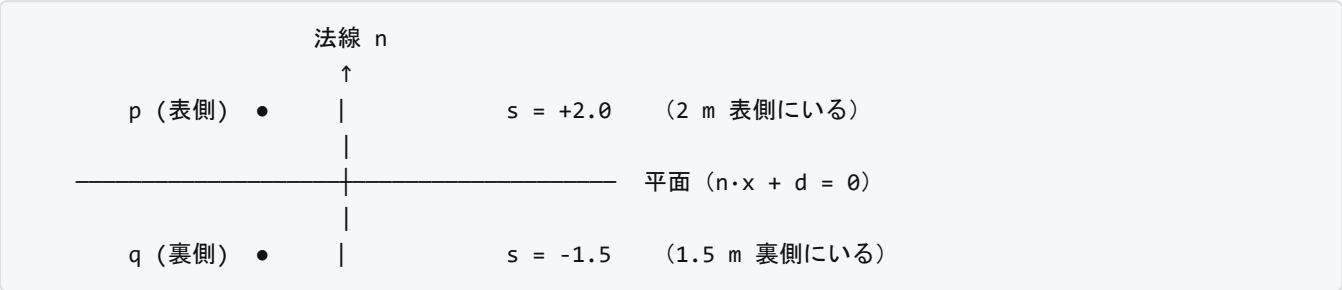
$$s = \mathbf{n}\cdot\mathbf{p} + d$$

を計算すると、これは「**点 p から平面までの距離**」に「**どちら側にいるか**」の符号がついた量になります。

- $s > 0$ … **p は法線が向いている側（表側）にいる**

- $s = 0$ … p は平面上にいる
- $s < 0$ … **p は法線と反対側（裏側）にいる**

これを**符号付き距離**と呼びます。技術説明書 5.2 節で交点を求めるときにも同じ量が出てきます。



1.2 鏡像の式

点 **p** の、この平面に関する鏡像 **p′** は

$$\boxed{\ \mathbf{p}' = \mathbf{p} - 2\,s\,\mathbf{n} \qquad (s = \mathbf{n}\cdot\mathbf{p} + d)\ }$$

なぜこれで鏡像になるのか：**p** を「平面に平行な成分」と「法線方向の成分」に分けると、 法線方向の成分の大きさがちょうど s です。鏡像とは「平面に平行な成分はそのまま、 法線方向の成分だけ符号を反転させる」操作なので、法線方向に $2s$ だけ戻せばよい、ということです。

数式で確かめると、$\mathbf{p}'$ の符号付き距離は

$$\mathbf{n}\cdot\mathbf{p}' + d = \mathbf{n}\cdot(\mathbf{p} - 2s\mathbf{n}) + d = (\mathbf{n}\cdot\mathbf{p} + d) - 2s\,(\mathbf{n}\cdot\mathbf{n}) = s - 2s = -s$$

となり、**符号だけがひっくり返って距離は同じ**。まさに鏡像です。 （$\mathbf{n}\cdot\mathbf{n} = 1$、つまり n が単位ベクトルであることを使いました。 n が単位ベクトルでない場合は $\mathbf{p}' = \mathbf{p} - 2\frac{s}{|\mathbf{n}|^2}\mathbf{n}$ です。）

---

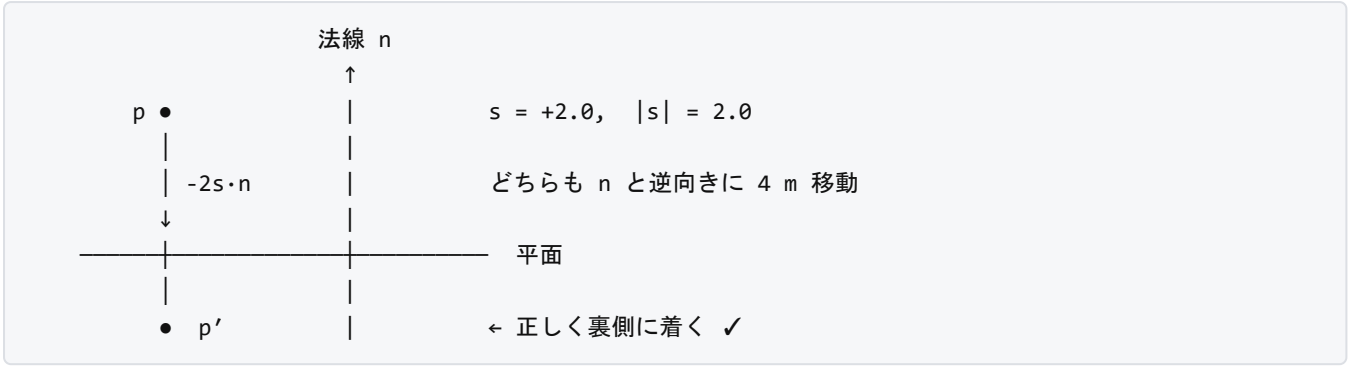
2. `abs()` を付けると何が起きるか

元コードは s の代わりに $|s|$ を使っています。

$$\mathbf{p}'_{\text{xxxxx}} = \mathbf{p} - 2\,|s|\,\mathbf{n}$$

2.1 表側にいる場合（$s > 0$）→ 一致する

$|s| = s$ なので、そもそも同じ式です。何も問題ありません。

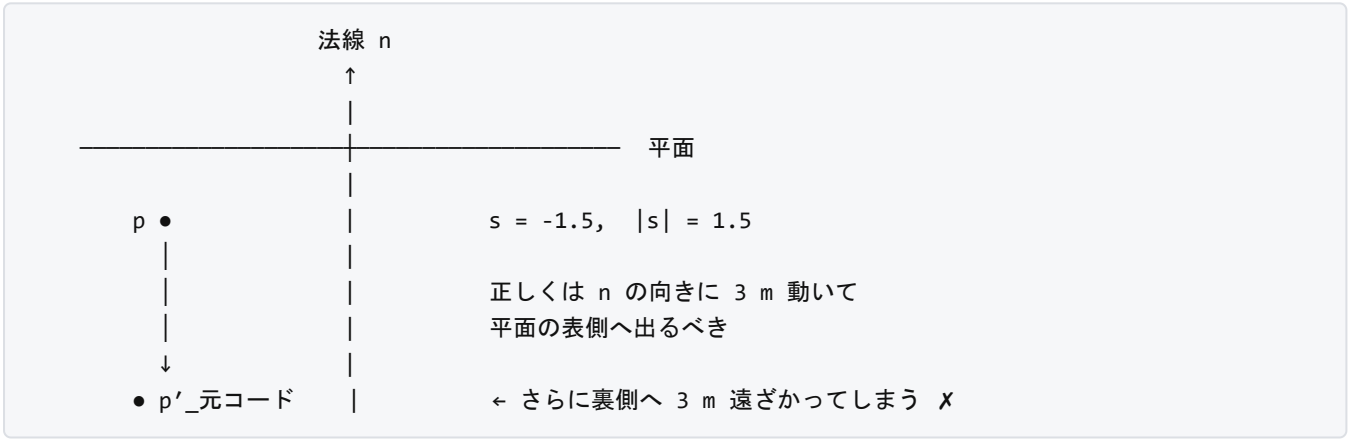


2.2 裏側にいる場合（$s < 0$）→ 壊れる

$|s| = -s$ なので、

$$\mathbf{p}'_{\text{壊れた}} = \mathbf{p} + 2s\mathbf{n}$$

となり、**正しい式と移動の向きが逆**になります。



正しい鏡像は平面の**反対側（表側）**に $|s|$ だけ離れた位置ですが、元コードは **同じ側（裏側）**に $3|s|$ 離れた位置へ飛びます。まったく別の点です。

---

3. では、実際に壊れるのか？

ここが判断の要点です。「**虚音源が面の裏側に来ることがあるのか**」という問題になります。

3.1 凸な部屋なら起きない

直方体のような**凸**な部屋を考えます。法線はすべて室内（空気側）を向いています。

- 実音源は室内にあるので、**すべての面について表側**（$s > 0$）にいます
- 1 回反射の虚音源は、反射した面については裏側に出ますが、

  その面をもう一度使うことはない（同じ面で連続 2 回反射する経路は存在しない）
- 次に鏡像を取る面については…

ここが微妙なところで、**凸な部屋なら 2 次以降の虚音源も、これから鏡像を取る面については 表側に留まる**ことが知られています。虚音源の位置は「部屋を鏡で折り返して並べた格子」の 上に乗るので、順序が正しい経路であれば常に表側から次の面を見込む形になります。

`test.dxf`（2×3×1 m の直方体）で `abs()` あり／なしを比較したところ、 **1 次・2 次反射とも完全に一致**しました。凸な部屋では実害がありません。

3.2 凹んだ形状・宙に浮いた物体では起きうる

問題が出るのは次のような場合です。

(a) 凹んだ壁（L 字の部屋、柱、袖壁など）

```
平面図（上から見た図）

┌──────────┐
│          │
│   音源●  │
│          │
│   ┌──────┘          ← 出っ張り（凹んだ形状）
│   │
│ A │  ← 面 A の法線は右向き（部屋側）
│   │     音源は面 A の表側にいる
└───┘

ところが「面 B → 面 A」の順に反射する経路を考えると、
面 B で折り返した虚音源が、面 A を含む無限平面の裏側に
回り込むことがある。
```

面は有限の三角形ですが、鏡像の計算は**無限平面**に対して行われます。 凹んだ形状では「面の広がりの外側」に虚音源が来ることがあり、 そのとき符号付き距離が負になりえます。

(b) 宙に浮いた物体（机、看板、反射板など）

厚みを持たせた物体の 2 面は互いに向き合っておらず、法線が逆を向いています。 物体の反対側にある虚音源は、当然その面の裏側にいます。

(c) 法線を反転させたモデル（`orient_normals='flip'`）

全部の法線を裏返すと、実音源からして**すべての面について裏側**になります。 この場合 `abs()` 版は**最初の 1 回目の鏡像から間違えます**。

3.3 間違えたらどうなるか

幸い、**間違った虚音源はバックトレースの検算で弾かれます**（技術説明書 7.2 節）。

バックトレースは受音点から虚音源へ直線を引き、 「最初に当たる面が経路の言う面と一致するか」を確認します。 虚音源の位置が違えば直線の向きも違うので、たいてい別の面に当たって却下されます。

つまり `abs()` の誤りは「**あるはずの経路を見落とす**」方向に効きます （偽の経路を作るのではなく、真の経路を落とす）。 音線法で見つけた経路が虚像法で軒並み却下される、という形で現れます。

過大評価ではなく過小評価なので気づきにくい、というのが厄介な点です。

---

4. 元コードがなぜ `abs()` を書いたのかの推測

確証はありませんが、次のどちらかだと思われます。

1. **「距離」という言葉に引きずられた**

   コメントに「音源位置と平面との距離を算出」とあり、距離は正の量なので `abs` を付けた。
   点と平面の距離の公式 $|n \cdot p + d|/|n|$ をそのまま持ってきたのだと思われます。
   この公式自体は正しいのですが、**鏡像に使うときは符号が必要**です。
2. **凸な部屋しか想定していなかった**

   実際に凸な部屋なら結果は同じなので、テストでは問題が出なかった。

なお元コードは `/ |n|` で割っています（917 行では $|n|^2$ で割るべきところ）。 これは法線を単位ベクトルに正規化してあるので結果的に問題ありません。 Python 版は正規化を明示的に行ってから計算しています。

---

5. 判断していただきたいこと

選択肢	内容	影響
(A) 現状のまま（符号付き）	数学的に正しい鏡像の式を使う	凸な部屋では元コードと同一。凹形状・浮いた物体・法線反転で正しく動くようになる
(B) 元コードに合わせる（`abs` あり）	元コードとビット単位で一致させることを優先	凹形状で経路を取りこぼす。`orient_normals='flip'` が使えなくなる
(C) 切り替え可能にする	引数で選べるようにする	元コードとの突き合わせ検証には便利だが、使う側が迷う

こちらの推奨は (A) です。理由:

- 凸な部屋では元コードと完全に一致するので、既存の検証結果は変わらない
- 実際のモデル（家具、袖壁、段差）は凹形状を含むのが普通
- 誤りの現れ方が「経路の取りこぼし」なので、間違っていても気づきにくい

元コードとの一致を厳密に確認したい局面があるなら (C) にしますが、 そのときも既定は (A) にしたいところです。

---

6. 確認したいこと（NEA 側で分かれば）

1. 元の Fortran は**凹んだ形状**で使った実績がありますか？

   （もし使っていて結果に不満がなかったなら、影響は小さいということになります）
2. `ynnmrev='y'`（法線全反転）を使ったことはありますか？

   これを使うと `abs()` 版は 1 次反射から破綻するので、
   使った実績があるなら「実は破綻していた」可能性があります。
3. 音線法で見つかった経路のうち、虚像法で却下される割合を記録していましたか？

   却下率が異常に高い事例があれば、この問題が原因かもしれません。

---

7. 検証に使えるコード

```
import numpy as np
import loop_noredundancy as ln

# 面の裏側にある点で違いを見る
normal = np.array([0.0, 0.0, 1.0])      # 上向き
plane_point = np.array([0.0, 0.0, 0.0])  # z = 0 の面
p = np.array([1.0, 1.0, -1.5])           # 面の裏側（z < 0）

d = -np.dot(normal, plane_point)
s = np.dot(normal, p) + d                # -1.5（負）

correct = p - 2 * s * normal             # [1, 1, +1.5] ← 正しい鏡像
fortran = p - 2 * abs(s) * normal        # [1, 1, -4.5] ← 元コード
print(correct, fortran)
```

`tests/test_geosim.py` の `test_image_sources` では

- 同じ面で 2 回鏡像を取ると元に戻る（対合性）
- 鏡像の前後で符号付き距離の符号が反転する

を確認しています。`abs()` 版はどちらも満たしません。